深度学习 chap4_ simple neural network 实验报告

2352473 蒋东珣

一、实验背景

本实验目标是在经典的 MNIST 手写数字识别任务上，用一个结构足够简单但能体现深度学习核心流程的两层全连接网络（输入展平为 784 维、隐藏层 100 维并使用 ReLU 非线性、输出层 10 类）完成从原始像素到数字类别的监督学习建模，从而掌握数据预处理、前向计算、损失函数设计、反向传播与参数更新、以及训练/测试评估等完整链路。

二、实验过程

2.1 Tensorflow 2.0 方案

2.1.1 函数定义

当前 tutorial_minst_fnn-tf2.0-exercise.ipynb 的核心函数围绕“数据读取—前向传播—损失/精度计算—训练更新”构成。数据侧用 mnist_dataset()封装了 MNIST 的读取与预处理流程：通过 tensorflow.keras.datasets.mnist.load_data() 得到训练集与测试集的 (x, y)，再将像素值除以 255.0 做归一化，使输入落在[0, 1]区间，从而让优化更稳定。模型侧用 forward(x)作为前向函数，它把输入的 28×28 灰度图用 tf.reshape(x, [-1, 28*28]) 展平为 784 维向量，然后依次完成第一层全连接与 ReLU 激活 h1 = relu(xW1 + b1)，再完成第二层全连接得到 logits = h1W2 + b2。训练指标侧定义了 compute_loss(logits, labels)，采用 tf.nn.sparse_softmax_cross_entropy_with_logits 直接对未归一化的 logits 计算稀疏交叉熵并取 batch 均值；精度函数 compute_accuracy(logits, labels)用 tf.argmax 得到预测类别并与真实标签逐样本比较后求平均，得到 0～1 的准确率。需要强调的是，我们同时保留了一个基于 tf.GradientTape() 的 train_one_step() 实现，但由于开头已经 tf.compat.v1.disable_eager_execution()关闭了 Eager，这段 Tape 写法在该运行方式下实际上不应参与训练流程；真正用于训练的是后面 Graph/Session 部分构建的 train_op = tf.compat.v1.train.AdamOptimizer(...).minimize(loss)，它在图中自动生成反向传播与参数更新算子，并由 Session.run 驱动执行。

2.1.2 数据采集

本实验使用 MNIST 手写数字数据集，包含 60000 张训练图片与 10000 张测试图片，每张图片为 28×28 的单通道灰度图，标签为 0～9 的整数类别。代码通过 datasets.mnist.load_data()一次性把数据加载到内存中，随后对图像做 x = x / 255.0 的数值缩放，避免原始 0～255 的像素范围导致梯度尺度过大、训练震荡等问题。在进入训练图之前，数据被显式转成 NumPy 的浮点/整型类型以匹配占位符约束：x_train/x_test 转为 np.float32，y_train/y_test 转为 np.int64，并在 Session.run(..., feed_dict={x_ph: x_train, y_ph: y_train}) 里通过 feed_dict 喂入计算图。由于当前实现是“整批（full-batch）训练”，每个 epoch 直接把全部 60000 张训练样本一次性送入图中计算一次梯度并更新一次参数，因此数据采集与喂入逻辑比较直接，但训练速度与显存/内存压力会显著高于小批量训练的写法。

2.1.3 模型描述

该模型是一个两层全连接神经网络（输入层不计入可学习层数），结构为 784→100→10。第一层参数为$W_1 \in R^{784\times100}$与$b_1 \in R^{100}$，将展平后的像素向量映射到 100 维隐层表征，并通过 ReLU 引入非线性，使网络具备拟合复杂决策边界的能力；第二层参数为$W_1 \in R^{100\times10}$与$b_2 \in R^{10}$，把隐层表征映射到 10 类的输出 logits。训练目标使用稀疏交叉熵 sparse_softmax_cross_entropy_with_logits，它在数值上等价于对 softmax(logits) 的负对数似然，但以 logits 形式计算能减少溢出/下溢风险并提升稳定性。优化器采用 Adam，学习率设为 1e-3，由 tf.compat.v1.train.AdamOptimizer(...).minimize(loss)在图中自动构建梯度并更新 W1、b1、W2、b2。运行模式方面，开头通过 tf.compat.v1.disable_eager_execution() 强制进入 TF1 兼容的 Graph 模式，训练阶段用 placeholder 表示输入输出，用 Session 执行图；这种方式的优点是能够完全绕开环境里 Eager/AutoGraph/estimator 相关的不兼容链路，代价是代码风格更接近 TF1，需要显式初始化变量并用 feed_dict 传数据。

2.1.4 拟合效果

拟合效果在 notebook 中通过每个 epoch 打印的 loss 与 accuracy 以及最终的 test loss / test accuracy 来呈现。按该网络规模与 MNIST 任务难度，正常情况下训练过程中应观察到损失逐步下降、训练准确率逐步上升，测试集准确率在若干轮后趋于稳定；如果损失长期不降或准确率接近随机猜测（约 0.1），通常意味着输入未正确归一化、标签 dtype/shape 与 sparse_softmax_cross_entropy_with_logits 不匹配，或图/变量未正确初始化。需要注意的是当前实现使用整批训练，每个 epoch 只有一次参数更新，因此相较于 mini-batch 训练，曲线可能呈现下降较慢但较平滑的特征；最终泛化性能也会受到学习率、epoch 数、隐藏层宽度以及是否采用批训练等因素影响。

```
epoch 0 : loss 2.5813822746276855 ; accuracy 0.12150000035762787
epoch 1 : loss 2.415104866027832 ; accuracy 0.15541666746139526
epoch 2 : loss 2.2823221683502197 ; accuracy 0.20784999430179596
epoch 3 : loss 2.173877239227295 ; accuracy 0.2589166760444641
epoch 4 : loss 2.0826680660247803 ; accuracy 0.30674999952316284
epoch 5 : loss 2.0031869411468506 ; accuracy 0.36434999108314514
epoch 6 : loss 1.9312512874603271 ; accuracy 0.4229666590690613
epoch 7 : loss 1.8637783527374268 ; accuracy 0.46131667494773865
epoch 8 : loss 1.7985143661499023 ; accuracy 0.48820000886917114
epoch 9 : loss 1.733949899673462 ; accuracy 0.5119166374206543
epoch 10 : loss 1.6692885160446167 ; accuracy 0.5387166738510132
epoch 11 : loss 1.604391098022461 ; accuracy 0.5664499998092651
epoch 12 : loss 1.5396288633346558 ; accuracy 0.5939666628837585
epoch 13 : loss 1.4757134914398193 ; accuracy 0.6216166615486145
epoch 14 : loss 1.413480520248413 ; accuracy 0.6461166739463806
epoch 15 : loss 1.353623390197754 ; accuracy 0.6682833433151245
epoch 16 : loss 1.2965996265411377 ; accuracy 0.6864666938781738
epoch 17 : loss 1.2425649166107178 ; accuracy 0.7001333236694336
epoch 18 : loss 1.1914747953414917 ; accuracy 0.7131666541099548
epoch 19 : loss 1.1430866718292236 ; accuracy 0.7238500118255615
epoch 20 : loss 1.0970796346664429 ; accuracy 0.7342000007629395
epoch 21 : loss 1.053175687789917 ; accuracy 0.7436666488647461
epoch 22 : loss 1.0112690925598145 ; accuracy 0.7533000111579895
epoch 23 : loss 0.9713420867919922 ; accuracy 0.7625666856765747
epoch 24 : loss 0.933436930179596 ; accuracy 0.7714166641235352
...
epoch 47 : loss 0.4965510070323944 ; accuracy 0.8676000237464905
epoch 48 : loss 0.4881874918937683 ; accuracy 0.8694833517074585
epoch 49 : loss 0.4802875220775604 ; accuracy 0.8712000250816345
test loss 0.457484632730484 ; accuracy 0.882099986076355
```

2.2 Numpy 方案

2.2.1 函数定义

这份 tutorial_minst_fnn-numpy-exercise.ipynb 的核心是用 NumPy 手工实现一套“可反向传播”的计算图原语，并在此基础上搭建两层全连接网络完成 MNIST 分类。最底层的算子类包括 Matmul、Relu、Softmax、Log：它们都采用同一种接口风格，forward(...) 负责前向计算并把必要的中间量缓存到 self.mem（例如 Matmul 缓存输入 x 和权重 W，Relu 缓存输入 x，Softmax 缓存输出概率 out，Log 缓存输入 x），backward(grad_y)则在已缓存的中间量基础上返回对输入的梯度（以及在 Matmul 中额外返回对权重的梯度）。例如 Matmul.backward 基于矩阵求导公式实现 grad_x = grad_y @ W.T 与 grad_W = x.T @ grad_y；Relu.backward 根据 x>0 的位置把上游梯度透传或置零；Softmax.backward 通过构造每个样本的雅可比（以 s_i s_j 的形式）实现 softmax 的反传；Log.backward 返回 grad_x = grad_y / x（加上 epsilon 防止除零）。在这些原语之上，myModel 封装了整网的前向与反向：forward(x) 完成输入展平、拼接偏置项、两次矩阵乘与一次 ReLU，并得到 softmax 概率与其 log；backward(label)则严格按计算链条从后往前依次调用各算子的 backward，得到 W2_grad 与 W1_grad。训练过程通过 train_one_step(model, x, y) 串起一次前向、一次反向、一次参数更新（W -= lr * grad），并用 compute_loss 与 compute_accuracy 输出当前拟合指标；测试过程 test(model, x, y)只做前向并计算指标。除训练主流程外，notebook 前半段还包含“Gradient check”与“Final Gradient Check”，通过把 NumPy 手写梯度与 TensorFlow 求导结果对照，验证自定义 backward 的正确性。

2.2.2 数据采集

数据仍然来自 MNIST，读取方式与 TF 版本类似：mnist_dataset() 调用 datasets.mnist.load_data()得到(x, y)与(x_test, y_test)，并把图像像素除以 255.0 归一化到[0,1]。与 TF Graph/Session 版本不同的是，这里训练标签不是直接用整数类别，而是显式构造 one-hot 矩阵：train_label 的形状为[N_train, 10]，test_label 的形状为 [N_test, 10]，通过 label[np.arange(N), y] = 1.0 将第 y 类置为 1，其余为 0。这样做与后续的损失形式匹配：损失采用 -sum(one_hot * log(prob)) 的交叉熵写法，要求标签与类别维度对齐。当前实现同样是“整批训练”，每个 epoch 把全部 60000 张训练集一次性送入 train_one_step，因此不会出现数据打乱、batch 切分等步骤，数据管线更简单，但每次前向/反向都在大矩阵上运算，对 CPU 时间与内存带宽要求更高。

2.2.3 模型描述

模型是一个两层全连接分类器，结构上等价于 Flatten(28×28) -> Dense(100) + ReLU -> Dense(10) -> Softmax，但用 NumPy 原语显式实现。参数部分有两组权重：W1 的形状是[28*28+1, 100]，这里的 +1 用于吸收偏置项（代码在 forward 里构造 bias = ones([N,1]) 并与输入拼接成[N, 785]，从而把第一层偏置合并进 W1）；W2 的形状是[100, 10]，第二层没有单独 bias，因此它是纯线性映射。前向传播的计算链路是：输入 x 展平为[N,784]，拼接 bias 后变为[N,785]，经 Matmul 得到隐层线性输出 h1，经 Relu 得到 h1_relu，再经第二次 Matmul 得到类别得分 h2，经 Softmax 得到概率 h2_soft，再经 Log 得到 h2_log（对每类概率取对数）。损失函数 compute_loss(log_prob, labels) 实现的是多类交叉熵的均值形式 mean(sum(-log_prob * one_hot, axis=1))，即只惩罚真实类别对应的-log p(y|x)；准确率 compute_accuracy(log_prob, labels) 通过 argmax 找到预测类与真实类并求平均，虽然传入的是 log_prob，但 log 是单调变换，argmax(log_prob)与 argmax(prob)完全一致，因此不会影响分类结果。反向传播遵循链式法则逐层回传：从 Log 的梯度开始，通过 Softmax、第二层 Matmul、Relu、第一层 Matmul，最终得到两层权重的梯度并用于参数更新。训练的学习率硬编码为 1e-5，属于比较保守的步长，优点是不容易发散，缺点是收敛可能较慢，尤其是在

整批训练且权重初始化为 np.random.normal 的情况下。

2.2.4 拟合效果

拟合效果由训练循环里每个 epoch 打印的 loss 与 accuracy 以及训练结束后在测试集上打印的 test loss 与 test accuracy 来体现。在正常实现下，随着 epoch 增加，应当观察到训练损失整体下降、训练准确率逐步上升，测试准确率在一定轮数后趋于稳定。由于当前写法是 full-batch，每个 epoch 只更新一次参数，数值变化往往比 mini-batch 更平滑但也更慢；同时学习率设为 1e-5，会进一步拉长“明显变好”的时间跨度，因此在报告中描述拟合效果时，更适合引用较长区间（例如前 10 轮与后 10 轮）的趋势对比，而不是只看相邻两三轮的波动。该实现还有两个会显著影响拟合表现与稳定性的细节：其一是 softmax 前向直接对 x 做 np.exp(x)，未做常见的 x - max(x)数值稳定化，在 logits 幅值较大时可能溢出导致 inf/NaN，从而损失和梯度异常；其二是第二层没有显式 bias，表达能力略弱但通常仍能在 MNIST 上得到可用准确率。若训练过程中出现 loss 变为 nan、accuracy 长期停留在约 0.1 或大幅震荡，这类现象通常与数值稳定性（softmax 溢出）、学习率/初始化尺度、以及整批训练导致的优化动力学有关；相反，如果输出能表现为“loss 逐步下降且 accuracy 明显高于 0.1”，即可作为模型确实学到了输入到类别映射的直接证据。报告里可以把训练日志按阶段摘录，并在最后给出测试集的量化指标作为最终拟合效果结论。

```
epoch 0 : loss 6.655560049932523 ; accuracy 0.7011666666666667
epoch 1 : loss 6.958603075134358 ; accuracy 0.6949833333333333
epoch 2 : loss 6.3873513913234214 ; accuracy 0.7130166666666666
epoch 3 : loss 6.533692863211798 ; accuracy 0.7102666666666667
epoch 4 : loss 6.4713674673069805 ; accuracy 0.7091333333333333
epoch 5 : loss 6.961274016600109 ; accuracy 0.6963166666666667
epoch 6 : loss 6.080168938185173 ; accuracy 0.7278166666666667
epoch 7 : loss 6.016412904044972 ; accuracy 0.73045
epoch 8 : loss 6.1004976080628435 ; accuracy 0.7262833333333333
epoch 9 : loss 6.338161632041869 ; accuracy 0.7183166666666667
epoch 10 : loss 6.14293180085913 ; accuracy 0.7249333333333333
epoch 11 : loss 6.3125251288960715 ; accuracy 0.71965
epoch 12 : loss 5.986707843744444 ; accuracy 0.7324666666666667
epoch 13 : loss 6.0355183151925464 ; accuracy 0.7302666666666666
epoch 14 : loss 5.844354955582449 ; accuracy 0.7390833333333333
epoch 15 : loss 5.862975655159635 ; accuracy 0.7375333333333334
epoch 16 : loss 5.716873179736476 ; accuracy 0.7453666666666666
epoch 17 : loss 5.70825930245691 ; accuracy 0.7443166666666666
epoch 18 : loss 5.6147831374184785 ; accuracy 0.74955
epoch 19 : loss 5.600104294107392 ; accuracy 0.7485166666666667
epoch 20 : loss 5.53285210625847 ; accuracy 0.7537166666666667
epoch 21 : loss 5.498229621218681 ; accuracy 0.7535333333333334
epoch 22 : loss 5.4481026533806185 ; accuracy 0.7571166666666667
epoch 23 : loss 5.414835228819008 ; accuracy 0.7570166666666667
epoch 24 : loss 5.389011437530993 ; accuracy 0.75975
...
epoch 97 : loss 4.460069720784722 ; accuracy 0.7991833333333334
epoch 98 : loss 4.458757514039832 ; accuracy 0.8009333333333334
epoch 99 : loss 4.445921770205131 ; accuracy 0.7998333333333333
test loss 4.47473378928673 ; accuracy 0.801
```

2.3 TensorFlow 图模式方案

2.3.1 函数定义

整体是一个“可直接运行的 MNIST 两层全连接分类器”，采用 TensorFlow 1.x 图模式（Graph + Session）来完成训练与评估，从而规避在 TF1 环境中使用 Eager/AutoGraph/GradientTape 可能触发的兼容性问题。文件最上方显式调用了 tf.compat.v1.disable_eager_execution()，这会让后续所有 TensorFlow 运算以静态计算图的方式构建，并在 tf.compat.v1.Session() 中执行。代码主体由四个关键函数/结构组成：_download(url, dst_path)负责把 MNIST 的 npz 文件下载到本地缓存目录并保证可重复使用；mnist_dataset(data_dir) 负责从本地 npz 读取原始数据并完成 dtype 与归一化处理；build_model(x_flat, hidden)负责在图中创建/复用两层全连接网络的参数并输出 logits；main()则负责组织完整的训练图、评估图与训练循环（包含每个 epoch 的训练统计和测试统计），最后在__main__入口中执行。

build_model 的实现方式是 TF1 典型写法：使用 tf.compat.v1.variable_scope("fnn", reuse=tf.compat.v1.AUTO_REUSE) 与 tf.compat.v1.get_variable(...) 创建权重与偏置，这里 AUTO_REUSE 的含义是：第一次调用时创建变量，后续在相同 scope 下再次调用时自动复用同名变量。正因为如此，脚本可以分别构建“训练图”和“评估图”，但两者共享同一套 W1/b1/W2/b2 参数，保证测试时用的就是训练得到的权重而不是一套新变量。网络计算本身很直接：h1 = relu(xW1 + b1)，logits = h1W2 + b2，返回的 logits 是未经过 softmax 归一化的类别得分，配合 tf.nn.sparse_softmax_cross_entropy_with_logits 使用可以获得更好的数值稳定性。

2.3.2 数据采集

数据采集完全通过本地缓存的 mnist.npz 来完成，并且实现了“缺失则下载、存在则复用”的逻辑。mnist_dataset(data_dir="mnist_data")先把目标路径设为 mnist_data/mnist.npz，再调用 _download(KERAS_MNIST_URL, npz_path)：如果文件已经存在且大小大于 0，就直接返回不再下载；否则从 https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz 拉取数据写入本地。这种方式的好处是：第一次运行需要联网，之后每次运行都不再依赖网络，数据也不会因为在线接口变化而影响实验复现。

读取数据时使用 np.load(npz_path)直接得到 x_train, y_train, x_test, y_test。原始 x_*的 dtype 为 uint8，值域在 0 到 255；代码把 x_train/x_test 转为 float32 并除以 255.0，将值域缩放到[0, 1]，这是训练全连接网络的标准预处理之一，能够显著改善梯度尺度和收敛稳定性。标签 y_*也被统一转成 int64，并保持为形状(N,)的类别索引而不是 one-hot，这与后续损失函数 sparse_softmax_cross_entropy_with_logits(labels=..., logits=...)的接口完全匹配。

训练时的数据喂入并非 feed_dict 全量灌入，而是用 tf.data 在图内构造输入管线：Dataset.from_tensor_slices((train_x, train_y))把 NumPy 数组转成数据集，shuffle(60000)做随机打乱，repeat()让数据集无限循环，batch(batch_size)形成批数据，prefetch(1)进行简单预取以减少输入阻塞。随后通过 make_one_shot_iterator(...).get_next()得到每一步训练的 xb_img 和 yb，从而训练阶段完全不需要 feed_dict，运行时每个 step 直接 sess.run([train_op, loss_b, acc_b])就能取到一批数据并更新一次参数。

2.3.3 模型描述

模型是一个标准的两层全连接前馈网络（FNN/MLP），用于 MNIST 十分类任务。输入为单张 28×28 灰度图，在训练图中由 xb_img 提供形状 (B, 28, 28)，随后用 tf.reshape(xb_img, [-1, 28 * 28])展平为(B, 784)的向量。第一层参数 W1 形状为(784, hidden)，偏置 b1 形状为(hidden,)；第二层参数 W2 形状为(hidden, 10)，偏置 b2 形状为(10,)。隐藏层激活函数使用 ReLU，这使得该网络具备非线性表达能力；输出层不做 softmax，而是输出 logits，把 softmax

融入到交叉熵里由 sparse_softmax_cross_entropy_with_logits 统一处理，这是深度学习框架中最常见、也最数值稳定的写法。

优化目标为 batch 平均交叉熵：loss_b = mean(CE(yb, logits_b))。训练准确率用 pred_b = argmax(logits_b)得到预测类别，再与 yb 比较计算平均正确率。优化器选用 Adam，学习率 lr=1e-3，通过 optimizer.minimize(loss_b)在图中自动构建反向传播并对所有可训练变量求梯度与更新。训练过程按 epochs=10 迭代，每个 epoch 内部执行 steps_per_epoch = ceil(60000 / batch_size) 次更新，这种“mini-batch+多步更新”的方式相比整批训练通常收敛更快、泛化更稳定，也更贴近真实深度学习训练实践。

评估（test）部分为了复用同一套参数，单独建立了 x_ph/y_ph 占位符路径：把 x_ph reshape 成(N,784)后同样调用 build_model(...) 得到 logits_eval，并计算 loss_eval/acc_eval。由于评估不需要反向图，也不需要 tf.data 管线，因此采用 feed_dict 分 batch 喂入测试集；测试指标使用“按 batch 大小加权平均”的方式聚合，这比直接对每个 batch 的平均值再平均更准确，尤其当最后一个 batch 的大小小于 batch_size 时，加权平均能避免最后一批对总体指标产生不合理的放大/缩小。

2.3.4 拟合效果

该脚本把“拟合效果”直接体现在每个 epoch 打印的一行日志中，格式为：epoch xx | train loss ... acc ... | test loss ... acc ...。从指标含义上看，train loss 反映模型对训练集当前批分布的负对数似然水平，通常会随着训练推进而下降；train acc 反映训练集上的分类正确率，通常会逐步上升。test loss/test acc 则是对测试集的泛化评估，用于判断模型是否真正学到了可迁移的判别边界，而不是仅在训练集上拟合。

在正常的数据归一化、损失定义与优化器设置下，这个 784→100→10 的网络在 MNIST 上应当能够在若干 epoch 内获得明显高于随机猜测（0.1）的准确率，并且测试准确率会随训练提升后逐渐趋稳。该实现通过 variable_scope+AUTO_REUSE 和 global_variables_initializer()已经覆盖了最常见的图构建陷阱，因此从“可训练可评估”的完整性上更接近一个可靠的基线实现。报告写作时，如果需要更“可视化”的拟合效果呈现，可以把每个 epoch 的 train loss/acc 与 test loss/acc 记录到列表并绘制曲线；但即使不画图，保留终端输出日志并在报告中摘录前期与末期若干 epoch 的对比，也能清晰证明模型确实在逐步收敛并提升泛化性能。

```
(tf1_env) PS C:\Users\HUAWEI\Desktop\SACB_Net-main\SACB_Net-main> python try.py
epoch 01 | train loss 0.5421 acc 0.8487 | test loss 0.2544 acc 0.9286
epoch 02 | train loss 0.2249 acc 0.9360 | test loss 0.1858 acc 0.9473
epoch 03 | train loss 0.1688 acc 0.9520 | test loss 0.1500 acc 0.9568
epoch 04 | train loss 0.1368 acc 0.9611 | test loss 0.1297 acc 0.9633
epoch 05 | train loss 0.1136 acc 0.9677 | test loss 0.1163 acc 0.9662
epoch 06 | train loss 0.0978 acc 0.9722 | test loss 0.1073 acc 0.9674
epoch 07 | train loss 0.0852 acc 0.9753 | test loss 0.0975 acc 0.9710
epoch 08 | train loss 0.0745 acc 0.9788 | test loss 0.0925 acc 0.9711
epoch 09 | train loss 0.0658 acc 0.9809 | test loss 0.0875 acc 0.9745
epoch 10 | train loss 0.0594 acc 0.9828 | test loss 0.0825 acc 0.9750
```

三、总结与心得

这三份代码的联系在于都围绕同一个学习目标：用一个简单的两层全连接网络去完成 MNIST 手写数字的十分类任务。三者在数学层面做的事情高度一致：输入是 28×28 的灰度图（通常会先展平为 784 维向量并做 0～1 归一化），经过一层线性变换再接 ReLU 得

到隐藏表示，再经过第二层线性变换得到 10 维输出得分（logits 或其对应的概率），用交叉熵损失衡量预测与真实标签的差异，通过梯度下降类方法更新参数，使训练损失下降、准确率上升。

tutorial_minst_fnn-numpy-exercise.ipynb 的核心特征是“手写反向传播”。我们用 NumPy 搭了一套最小可用的自动微分思想：每个算子类都有 forward 与 backward，前向时缓存必要中间量，反向时用链式法则把上游梯度一层层传回去，最终得到 W1_grad/W2_grad，再用 W -= lr * grad 更新权重。它把标签转换成 one-hot 向量，损失用-sum(one_hot * log(prob))的形式显式写出来，因此最能体现“softmax、log、交叉熵、矩阵乘、ReLU 的梯度到底怎么推出来、怎么实现”。同一个 notebook 里出现的 TensorFlow 相关内容主要用于 gradient check，也就是把手写的梯度与框架算出来的梯度做对照验证；在这份代码里，TensorFlow 更像验算器，并不是训练主引擎。

tutorial_minst_fnn-tf2.0-exercise.ipynb 在机制上则是框架自动求导 + 图模式执行。它通过 tf.compat.v1.disable_eager_execution()强制进入 TF1 风格的静态图，前向由 forward(x) 定义，损失由 tf.nn.sparse_softmax_cross_entropy_with_logits 计算，参数更新由 tf.compat.v1.train.AdamOptimizer(...).minimize(loss) 自动构建反向图并更新变量，训练由 Session.run(..., feed_dict=...) 驱动。它和 NumPy 版的本质区别不在数学目标，而在于“梯度的产生方式”：NumPy 版梯度由手写的 backward 给出；TF 图模式版梯度由 TensorFlow 在图里自动生成。需要特别指出的是，我们仍保留了一段基于 tf.GradientTape()的 train_one_step（Eager 风格）函数，但在关闭 eager 的前提下这段逻辑不应该再参与训练，当前真正起作用的是后面那段 placeholder+Session 的训练图。

numpy_2.py 和上一份 TF 图模式 notebook 在训练机制上同源，都是 TF1 图模式、都是 minimize(loss) 自动求导更新参数、都是在 Session 中运行；但它更像一个完整的可复现训练脚本。它自己实现了 MNIST 的下载与本地缓存（从固定 URL 拉取 mnist.npz，存在则复用），避免因环境中 Keras/TF 数据接口变化导致的数据读取问题；它用 tf.data 构建 mini-batch 输入管线（shuffle、repeat、batch、prefetch），使每个 epoch 内有很多 step 更新，训练效率与收敛形态更接近标准深度学习实践；它还用 variable_scope(..., AUTO_REUSE) 明确保证训练图与评估图复用同一套变量，这让训练时更新的权重与测试时使用的权重天然一致，工程上更稳健。相较之下，TF notebook 版本用的是 feed_dict 整批喂入，通常每个 epoch 只更新一次，速度与收敛都更演示型，而不是标准训练型。

把三者放在一起看，它们之间最重要的对应关系是：NumPy 版显式写出的那条反向传播链（Log→Softmax→Matmul→Relu→Matmul）在 TensorFlow 版里是被 minimize(loss) 隐式生成的；两者的“联系”不是代码复用，而是数学等价。也正因为这种等价，我们才能用 TensorFlow 做 gradient check 来验证 NumPy 的 backward 是否正确。再进一步，从训练策略角度看，numpy_2.py 更像把 TF notebook 版升级为 mini-batch+数据管线+可复现数据下载的工程化实现。